Oracle DBA

Hands-On Practical Guide

Beginner → Intermediate → Advanced

---

20 Step-by-Step Labs | Oracle 19c | Linux Environment

Real Commands | Real Scenarios | Real Skills

🐧 **Linux Setup**	⚙️ **Install Oracle**	🔐 **Users & Security**	💾 **Backup & RMAN**
📊 **Performance**	🔄 **Data Guard**	**RAC Basics**	🔧 **Patching**

April 2026 | Oracle Database 19c

How to Use This Practical Guide

This guide contains 20 hands-on labs. Each lab gives you real commands to run on a real Oracle 19c database. Do NOT just read — TYPE every command yourself. Muscle memory is how DBAs learn.

🖥️ What You Need Before Starting Labs

Requirement	Details
Your PC/Laptop	Windows 10/11 or macOS — 8 GB RAM minimum, 16 GB recommended, 100 GB free disk
VirtualBox	Free from virtualbox.org — version 6.1 or higher. This lets you run a Linux server on your PC.
Oracle Linux 7 or 8	Free from oracle.com/linux. This is the OS for your virtual server.
Oracle 19c Software	Free download from oracle.com/database — 'LINUX.X64_193000_db_home.zip' (~3 GB)
PuTTY (Windows only)	Free SSH terminal from putty.org — to connect to your virtual Linux server
Internet connection	Needed to download software and Oracle Linux updates (yum packages)
6–8 hours total time	These 20 labs take about 6–8 hours across multiple sessions. Do not rush.

💡 **TIP:** If you do not have a laptop with enough RAM, use Oracle Cloud Free Tier (cloud.oracle.com) — you get a free VM with Oracle Linux already installed. No VirtualBox needed!

📋 Lab Roadmap — All 20 Labs at a Glance

Lab #	Lab Name	Phase	Level
01	Set Up Your Lab Environment (VirtualBox + Oracle Linux)	Phase 1: Setup	**Beginner**
02	Install Oracle 19c Database Software on Linux	Phase 1: Setup	**Beginner**
03	Create Your First Oracle Database Using DBCA	Phase 1: Setup	**Beginner**

04	Start, Stop & Connect to the Oracle Database	Phase 1: Setup	**Beginner**
05	Explore Oracle Architecture — Files, Memory & Processes	Phase 1: Setup	**Beginner**
06	Create & Manage Users, Roles, and Privileges	Phase 2: Admin	**Beginner**
07	Create & Manage Tablespaces and Datafiles	Phase 2: Admin	**Beginner**
08	Work with Tables, Indexes, and SQL Queries	Phase 2: Admin	**Beginner**
09	Manage Redo Logs and Archive Log Mode	Phase 2: Admin	**Intermediate**
10	Control Files — Backup, Restore, Multiplex	Phase 2: Admin	**Intermediate**
11	Configure RMAN — First Backup & Recovery	Phase 3: Backup	**Intermediate**
12	RMAN Incremental Backup & Point-in-Time Recovery	Phase 3: Backup	**Intermediate**
13	Oracle Flashback — Table & Database Recovery	Phase 3: Backup	**Intermediate**
14	Data Pump Export (expdp) and Import (impdp)	Phase 3: Backup	**Intermediate**
15	Performance Tuning — AWR, ADDM & Wait Events	Phase 4: Tuning	**Intermediate**
16	SQL Performance — Execution Plan & Index Tuning	Phase 4: Tuning	**Intermediate**
17	User Sessions, Locks, and Blocking Sessions	Phase 4: Tuning	**Intermediate**
18	Apply Oracle Patch Using OPatch	Phase 5: Advanced	**Advanced**
19	Set Up Data Guard (Physical Standby)	Phase 5: Advanced	**Advanced**
20	Monitor the Database — Alert Log, OEM & Scripts	Phase 5: Advanced	**Advanced**

PHASE 1

Environment Setup & Oracle Installation

Labs 01 – 05 | Estimated Duration: 3–4 Hours

🔬 LAB 01 Set Up Your Lab Environment (VirtualBox + Oracle Linux)

⏱ Time: 60–90 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Install VirtualBox on your PC and create a virtual Linux server (Oracle Linux 7 or 8) that will host Oracle Database 19c.

🎯 **WHY THIS MATTERS:** Every Oracle DBA works on Linux in production. Without a practice server you cannot run any commands. This lab gives you your own personal Oracle Linux server for free.

🏆 **WHAT YOU WILL LEARN:** How to create and configure a Linux virtual machine ready for Oracle installation.

Part A — Install VirtualBox

STEP 1: Download VirtualBox
Open your browser and go to: https://www.virtualbox.org/wiki/Downloads
Download 'VirtualBox for Windows hosts' (or macOS if you are on Mac).
Also download the 'VirtualBox Extension Pack' from the same page.
Install VirtualBox by running the downloaded file. Accept all defaults.
After install, double-click Extension Pack file to install it inside VirtualBox.

STEP 2: Download Oracle Linux 8.9 ISO
Go to: https://yum.oracle.com/oracle-linux-isos.html
Download: OracleLinux-R8-U9-x86_64-dvd.iso (about 10 GB)
This is the installation DVD image for Oracle Linux 8.
Save it somewhere you can find it easily (e.g. D:\OracleLinux\)

STEP 3: Create the Virtual Machine
Open VirtualBox → Click 'New'
Name: oracle-db Type: Linux Version: Oracle Linux (64-bit)

Memory: Set to 4096 MB (4 GB minimum — 8 GB recommended if you have it)
Hard Disk: Create new → VDI → Dynamically allocated → Size: 80 GB
Click Create.

STEP 4: Configure VM Settings
Right-click your new VM → Settings
System → Processor → Set CPUs to 2
Storage → Click empty DVD icon → Choose your Oracle Linux ISO file
Network → Adapter 1 → Set to 'NAT'
Network → Adapter 2 → Enable → Set to 'Host-Only Adapter'
Click OK.

STEP 5: Install Oracle Linux
Start the VM (click Start button)
Select 'Install Oracle Linux 8.9'
Language: English → Continue
Installation Destination: Click the disk → Click Done (auto partitioning)
Network: Turn ON the eth0 adapter → Set hostname to: oracle-db.localdomain
Software Selection: Choose 'Server with GUI' as base environment
Set root password: Welcome1# (for your practice lab only)
Create user 'oracle' with password: Oracle123!
Click Begin Installation → Wait 15–20 minutes
Reboot when finished.

📝 **NOTE:** After install, run: sudo yum update -y to apply all latest updates. This takes 10–15 minutes.

✅ Lab Completion Checklist

☑ VirtualBox is installed on your PC
☑ Oracle Linux VM is created with 4 GB RAM and 80 GB disk
☑ You can log in to Oracle Linux as root and oracle user
☑ Network is configured (you can ping google.com from the VM)

🔬 LAB 02 Install Oracle 19c Database Software on Linux

⏱ Time: 60–90 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Install the Oracle 19c Database software onto your Oracle Linux server. This installs the Oracle programs but does NOT create a database yet.

🎯 **WHY THIS MATTERS:** Installing Oracle software is the first thing every new DBA does. You must know every step by heart for interviews and real jobs.

🏆 **WHAT YOU WILL LEARN:** Linux prerequisite setup, Oracle directory structure, environment variables, and running the Oracle installer (OUI).

Part A — Download Oracle 19c Software

Go to oracle.com/database → Downloads → Oracle Database 19c

Download: LINUX.X64_193000_db_home.zip (approximately 3 GB)

You need a free Oracle account to download. Register at oracle.com if needed.

Part B — OS Prerequisites (run as root)

Open a terminal. Switch to root: su - root

Step 1 — Install the preinstall RPM (does most of the setup automatically):

```
# This single command creates oracle user, groups, and sets kernel
parameters
yum install -y oracle-database-preinstall-19c


# Verify oracle user was created:
id oracle
# Expected output: uid=54321(oracle) gid=54321(oinstall)
groups=54321(oinstall),54322(dba)
```

Step 2 — Set oracle user password:

```
passwd oracle
# Type: Oracle123!  (twice to confirm)
```

Step 3 — Create Oracle directories:

```
mkdir -p /u01/app/oracle/product/19.0.0/dbhome_1
mkdir -p /u02/oradata
mkdir -p /u02/oracle/FRA
mkdir -p /backup
chown -R oracle:oinstall /u01 /u02 /backup
```

```
chmod -R 775 /u01 /u02
```

Step 4 — Disable firewall and set SELinux to permissive:

```
systemctl stop firewalld && systemctl disable firewalld
sed -i 's/SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config
setenforce 0
```

Step 5 — Update /etc/hosts file:

```
# Find your server IP first:
ip addr show | grep 'inet '
# Add to /etc/hosts (replace 192.168.56.101 with YOUR IP):
echo '192.168.56.101 oracle-db.localdomain oracle-db' >> /etc/hosts
```

Part C — Set Environment Variables (run as oracle user)

Switch to oracle user: su - oracle

Edit the bash profile: vi ~/.bash_profile

Add these lines at the bottom of the file:

```
export ORACLE_BASE=/u01/app/oracle
export ORACLE_HOME=$ORACLE_BASE/product/19.0.0/dbhome_1
export ORACLE_SID=ORCL
export PATH=$ORACLE_HOME/bin:$PATH
export LD_LIBRARY_PATH=$ORACLE_HOME/lib:/lib:/usr/lib
export CLASSPATH=$ORACLE_HOME/jlib:$ORACLE_HOME/rdbms/jlib
```

Save the file (:wq in vi) and apply it:

```
. ~/.bash_profile
echo $ORACLE_HOME   # Should show the path you set
```

Part D — Install Oracle Software

While still as oracle user, extract and run the installer:

```
# Copy the Oracle zip to ORACLE_HOME and extract it:
cd $ORACLE_HOME
# (Copy LINUX.X64_193000_db_home.zip here first using WinSCP or scp)
unzip -q LINUX.X64_193000_db_home.zip

# Run the Oracle installer:
```

```
./runInstaller
# A graphical installer window will open.
# Choose: 'Set Up Software Only' (do NOT create DB yet)
# Installation Type: Single Instance
# Oracle home user: Use Existing User → oracle
# Install Location: Already set to $ORACLE_HOME
# Click Install → Wait 10-15 minutes
```

⚠️ **WARNING:** When installer says 'Execute Configuration Scripts', open a NEW terminal as ROOT and run the two scripts it shows. Then click OK in the installer.

```
# Run as ROOT when installer prompts:
/u01/app/oraInventory/orainstRoot.sh
/u01/app/oracle/product/19.0.0/dbhome_1/root.sh
# Press Enter when asked for local bin directory (accept default
/usr/local/bin)
```

✅ Lab Completion Checklist

- ☑ Oracle 19c software is installed in /u01/app/oracle/product/19.0.0/dbhome_1
- ☑ Environment variables ORACLE_HOME, ORACLE_BASE, ORACLE_SID are set
- ☑ The oracle user can run: sqlplus -V (shows Oracle version)
- ☑ Oracle installer completed with 'Successfully Setup Software' message

🔬 LAB 03 Create Your First Oracle Database Using DBCA

⏱ Time: 45–60 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Use Oracle's DBCA (Database Configuration Assistant) tool to create your first Oracle 19c database named ORCL.

🎯 **WHY THIS MATTERS:** Installing Oracle software and creating a database are two separate steps. DBCA is the standard tool for creating databases in Oracle environments.

🏆 **WHAT YOU WILL LEARN:** How to use DBCA, what options to choose, CDB vs non-CDB architecture, and how to verify the database was created.

Step 1 — Start the Oracle Listener First

```
# As oracle user:
lsnrctl start
```

```
lsnrctl status
# Look for: 'The command completed successfully'
```

Step 2 — Launch DBCA

```
# As oracle user, run DBCA (graphical tool):
dbca
# If GUI does not open: export DISPLAY=:0.0  then try again
```

Step 3 — DBCA Wizard Settings

Follow these choices in the DBCA wizard:

- Step 1 of DBCA: Select 'Create a database'
- Step 2: Choose 'Typical Configuration' (easiest for beginners)
- Global Database Name: ORCL
- Storage Type: File System
- Database File Location: /u02/oradata
- Fast Recovery Area: /u02/oracle/FRA — Size: 10 GB
- Database Type: General Purpose
- IMPORTANT: Check 'Create as Container Database (CDB)' — PDB name: orclpdb
- Character Set: AL32UTF8 (Unicode — supports all languages)
- SYS password: SysPass123! SYSTEM password: SysPass123!
- Click Finish → Wait 20–30 minutes for database creation

💡 **TIP:** CDB (Container Database) is the modern Oracle architecture since 12c. One CDB can hold many PDBs (Pluggable Databases). Always choose CDB for new databases.

Step 4 — Verify the Database Was Created

```
# Connect to the database as SYSDBA:
sqlplus / as sysdba

-- Check database status:
SELECT name, db_unique_name, open_mode, log_mode FROM v$database;
-- Expected: ORCL  ORCL  READ WRITE  NOARCHIVELOG

-- Check instance status:
SELECT instance_name, status, startup_time FROM v$instance;

-- List all PDBs:
SELECT con_id, name, open_mode FROM v$pdbs;
```

```
-- Should show: ORCLPDB

-- Check database files exist:
SELECT name FROM v$datafile;

EXIT
```

✅ Lab Completion Checklist

☑ Oracle database ORCL is created and in OPEN / READ WRITE state
☑ PDB named ORCLPDB is created inside the CDB
☑ Listener is running and listening on port 1521
☑ You can connect with: sqlplus / as sysdba

🔬 LAB 04 Start, Stop & Connect to Oracle Database

⏱ Time: 30–45 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Practice starting and stopping the Oracle database and listener using SQL*Plus and Linux commands. Practice different connection methods.

🎯 **WHY THIS MATTERS:** DBAs start and stop databases for maintenance, patching, and emergencies. These are the most fundamental commands you will use every single day.

🏆 **WHAT YOU WILL LEARN:** Database startup modes (NOMOUNT, MOUNT, OPEN), shutdown modes, listener control, and multiple ways to connect.

Part A — Starting the Database

```
# Connect as SYSDBA:
sqlplus / as sysdba

-- Check current status:
SELECT status FROM v$instance;

-- If database is down, start it:
STARTUP;

-- The startup goes through 3 stages automatically:
```

```
-- Stage 1: NOMOUNT  -- Instance starts, memory allocated, background
processes start
-- Stage 2: MOUNT    -- Control file is read, database is mounted
-- Stage 3: OPEN     -- Data files opened, database available to users


-- You can also start stage by stage (useful for recovery):
-- STARTUP NOMOUNT;   -- Only start instance
-- ALTER DATABASE MOUNT;  -- Mount the database
-- ALTER DATABASE OPEN;   -- Open the database
```

Part B — Stopping the Database

```
-- Option 1: SHUTDOWN NORMAL (waits for all users to disconnect -- can
take long)
SHUTDOWN NORMAL;


-- Option 2: SHUTDOWN IMMEDIATE (kicks out users, rolls back transactions
-- most common)
SHUTDOWN IMMEDIATE;


-- Option 3: SHUTDOWN ABORT (emergency only -- like pulling power plug,
needs recovery on restart)
SHUTDOWN ABORT;


-- Option 4: SHUTDOWN TRANSACTIONAL (waits for current transactions to
finish)
SHUTDOWN TRANSACTIONAL;
```

💡 **TIP:** In production: always use SHUTDOWN IMMEDIATE. It is clean and fast. Never use SHUTDOWN ABORT unless it is an emergency — the database will need crash recovery on next startup.

Part C — Managing the Listener

```
# Exit SQL*Plus first. Back at Linux prompt:
EXIT


# Start listener:
lsnrctl start


# Check listener status:
lsnrctl status
```

```
# Stop listener:
lsnrctl stop

# Reload listener config without stopping:
lsnrctl reload
```

Part D — Different Ways to Connect

```
# 1. Connect locally as SYSDBA (no password needed -- OS authentication):
sqlplus / as sysdba

# 2. Connect with username and password:
sqlplus system/SysPass123!

# 3. Connect via TNS (network connection -- same as applications use):
sqlplus system/SysPass123!@localhost:1521/ORCL

# 4. Connect to the PDB:
sqlplus system/SysPass123!@localhost:1521/orclpdb

# 5. From inside SQL*Plus, switch to PDB:
ALTER SESSION SET CONTAINER = ORCLPDB;
SHOW CON_NAME;   -- Should show: ORCLPDB
```

✅ Lab Completion Checklist

☑	Can start database using STARTUP and STARTUP NOMOUNT/MOUNT/OPEN
☑	Can stop database using SHUTDOWN IMMEDIATE
☑	Can start and stop the listener with lsnrctl
☑	Can connect locally and via network (TNS) to both CDB and PDB

🔬 LAB 05 Explore Oracle Architecture — Files, Memory & Processes

⏱ Time: 45 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Query Oracle's dynamic performance views (V$ views) to see the actual memory areas, background processes, and database files of YOUR running database.

🎯 **WHY THIS MATTERS:** Theory is good but seeing real numbers from your own database makes architecture concepts stick. This lab turns theory into real hands-on evidence.

🏆 **WHAT YOU WILL LEARN:** How to read V$ views to see SGA memory, PGA, background processes, data files, redo logs, and control files.

Part A — Explore Memory (SGA & PGA)

```
sqlplus / as sysdba

-- See total SGA size:
SELECT name, ROUND(value/1024/1024,2) AS size_mb FROM v$sga;

-- See SGA components in detail:
SELECT component, ROUND(current_size/1024/1024,2) AS current_mb
FROM v$sga_dynamic_components
WHERE current_size > 0;

-- See PGA statistics:
SELECT name, ROUND(value/1024/1024,2) AS mb FROM v$pgastat
WHERE name IN ('total PGA allocated','maximum PGA allocated');
```

Part B — See Background Processes

```
-- List all Oracle background processes:
SELECT pname, description FROM v$bgprocess WHERE paddr <> '00'
ORDER BY pname;

-- Find the key ones: DBWR, LGWR, SMON, PMON, CKPT
-- You should see them all running!

-- Also check at OS level:
-- (Open a new terminal, as oracle user):
ps -ef | grep ora_ | grep -v grep
-- You will see processes like: ora_dbw0_ORCL, ora_lgwr_ORCL,
ora_smon_ORCL
```

Part C — View Physical Database Files

```
-- List all data files:
SELECT file#, name, status, ROUND(bytes/1024/1024,0) AS size_mb FROM
v$datafile;

-- List redo log files:
SELECT l.group#, l.status, l.members, lf.member AS filename
FROM v$log l JOIN v$logfile lf ON l.group# = lf.group#;

-- List control files:
SELECT name FROM v$controlfile;

-- See all from OS level (as oracle user in terminal):
ls -lh /u02/oradata/ORCL/
-- You will see the actual .dbf, .log, and .ctl files
```

Part D — Database & Instance Summary

```
-- One-stop info about your database:
SELECT name, db_unique_name, open_mode, log_mode,
       created, resetlogs_time FROM v$database;

-- Instance info:
SELECT instance_name, host_name, version,
       startup_time, status FROM v$instance;

-- CDB and PDB info:
SELECT con_id, name, open_mode, restricted FROM v$pdbs;
```

✅ Lab Completion Checklist

☑ Can see SGA memory components using V$SGA and V$SGA_DYNAMIC_COMPONENTS
☑ Can list all Oracle background processes from V$BGPROCESS and using 'ps' command
☑ Can see data files, redo logs, and control files using V$DATAFILE, V$LOG, V$CONTROLFILE
☑ Understand what each physical file does in the Oracle architecture

PHASE 2

Core Database Administration

Labs 06 – 10 | Estimated Duration: 3–4 Hours

🔬 LAB 06 Create & Manage Users, Roles, and Privileges

⏱ Time: 45–60 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Create database users, assign roles, grant and revoke object-level privileges, create profiles for password policies, and test access control.

🎯 **WHY THIS MATTERS:** User management is a daily DBA task. You will be asked about this in every interview and will do it every week in your job.

🏆 **WHAT YOU WILL LEARN:** CREATE USER, GRANT, REVOKE, CREATE ROLE, CREATE PROFILE, and how Oracle's privilege system works.

Part A — Create Users

```
sqlplus / as sysdba

-- Connect to the PDB first (where application users live):
ALTER SESSION SET CONTAINER = ORCLPDB;

-- Create an application developer user:
CREATE USER app_dev
  IDENTIFIED BY DevPass123!
  DEFAULT TABLESPACE USERS
  TEMPORARY TABLESPACE TEMP
  QUOTA 500M ON USERS;

-- Create a read-only reporting user:
CREATE USER reporter
  IDENTIFIED BY RepPass123!
  DEFAULT TABLESPACE USERS
  TEMPORARY TABLESPACE TEMP
  QUOTA 0 ON USERS;     -- No quota = cannot create tables
```

Part B — Grant Privileges

```
-- Grant basic login ability:
GRANT CREATE SESSION TO app_dev;
GRANT CREATE SESSION TO reporter;

-- Grant developer privileges:
GRANT CREATE TABLE, CREATE VIEW, CREATE PROCEDURE,
      CREATE SEQUENCE, CREATE TRIGGER TO app_dev;

-- Grant system role (bundle of privileges):
GRANT RESOURCE TO app_dev;

-- Create a sample table as app_dev for testing:
-- (Open a new connection as app_dev)
-- sqlplus app_dev/DevPass123!@localhost:1521/orclpdb
CREATE TABLE employees (
  emp_id    NUMBER PRIMARY KEY,
  emp_name  VARCHAR2(100),
  dept      VARCHAR2(50),
  salary    NUMBER(10,2)
);

INSERT INTO employees VALUES (1,'Rajesh Kumar','IT',75000);
INSERT INTO employees VALUES (2,'Priya Sharma','Finance',85000);
INSERT INTO employees VALUES (3,'Amit Patel','HR',65000);
COMMIT;
```

Part C — Object Privileges

```
-- Back as sysdba in ORCLPDB:
-- Give reporter SELECT access to app_dev's employees table:
GRANT SELECT ON app_dev.employees TO reporter;

-- Test: connect as reporter and try to select:
-- sqlplus reporter/RepPass123!@localhost:1521/orclpdb
SELECT * FROM app_dev.employees;   -- Should WORK
DELETE FROM app_dev.employees;     -- Should FAIL: insufficient privileges
```

Part D — Create Profile & Apply to User

```
-- As sysdba in ORCLPDB:
CREATE PROFILE app_profile LIMIT
  FAILED_LOGIN_ATTEMPTS  5
  PASSWORD_LIFE_TIME     90
  PASSWORD_REUSE_TIME    365
  SESSIONS_PER_USER      10
  IDLE_TIME              30;

-- Apply profile to users:
ALTER USER app_dev PROFILE app_profile;
ALTER USER reporter PROFILE app_profile;

-- Verify:
SELECT username, profile, account_status FROM dba_users
WHERE username IN ('APP_DEV','REPORTER');
```

Part E — Lock, Unlock, and Drop Users

```
-- Lock a user (no login allowed):
ALTER USER reporter ACCOUNT LOCK;

-- Unlock a user:
ALTER USER reporter ACCOUNT UNLOCK;

-- Reset password:
ALTER USER reporter IDENTIFIED BY NewRepPass123!;

-- Drop a user and ALL their objects:
DROP USER app_dev CASCADE;
-- CASCADE removes all tables, views, etc. owned by app_dev

-- Check users in the database:
SELECT username, account_status, created, profile
FROM dba_users WHERE oracle_maintained = 'N'
ORDER BY username;
```

⚠️ **WARNING:** DROP USER ... CASCADE is permanent and irreversible. Always confirm you are in the right database/PDB before dropping users in production!

✅ Lab Completion Checklist

- ☑ Created users with DEFAULT TABLESPACE, TEMP TABLESPACE, and QUOTA
- ☑ Granted system privileges (CREATE TABLE) and object privileges (SELECT ON table)
- ☑ Created a PASSWORD PROFILE and applied it to users
- ☑ Practised LOCK, UNLOCK, password reset, and DROP USER CASCADE

🔬 LAB 07 Create & Manage Tablespaces and Datafiles

⏱ Time: 45 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Create a new tablespace, add datafiles to it, resize it, monitor its usage, and practise adding space when it gets full.

🎯 **WHY THIS MATTERS:** Space management is the number-one daily DBA task. You will be called at night when tablespace is full. This lab teaches you exactly what to do.

🏆 **WHAT YOU WILL LEARN:** CREATE TABLESPACE, ADD DATAFILE, AUTOEXTEND, monitoring space with DBA views, and handling the 'tablespace full' scenario.

```
sqlplus / as sysdba
ALTER SESSION SET CONTAINER = ORCLPDB;

-- Create a new tablespace for application data:
CREATE TABLESPACE app_data
  DATAFILE '/u02/oradata/ORCL/ORCLPDB/app_data01.dbf'
  SIZE 500M
  AUTOEXTEND ON NEXT 100M MAXSIZE 5G
  EXTENT MANAGEMENT LOCAL
  SEGMENT SPACE MANAGEMENT AUTO;

-- Verify it was created:
SELECT tablespace_name, status, contents, extent_management
FROM dba_tablespaces
WHERE tablespace_name = 'APP_DATA';

-- Check the datafile:
SELECT file_id, file_name, ROUND(bytes/1024/1024,0) AS size_mb,
```

```
       autoextensible, ROUND(maxbytes/1024/1024/1024,1) AS maxsize_gb
FROM dba_data_files WHERE tablespace_name = 'APP_DATA';

-- Create a table in this new tablespace:
CREATE TABLE hr_data.big_table (id NUMBER, data VARCHAR2(100))
TABLESPACE APP_DATA;

-- Simulate tablespace getting full -- add a second datafile:
ALTER TABLESPACE app_data
  ADD DATAFILE '/u02/oradata/ORCL/ORCLPDB/app_data02.dbf'
  SIZE 1G AUTOEXTEND ON NEXT 256M MAXSIZE 10G;

-- Resize an existing datafile:
ALTER DATABASE DATAFILE '/u02/oradata/ORCL/ORCLPDB/app_data01.dbf'
  RESIZE 2G;

-- Check space usage in all tablespaces:
SELECT df.tablespace_name,
       ROUND(df.total_mb,0) AS total_mb,
       ROUND(df.total_mb - NVL(fs.free_mb,0),0) AS used_mb,
       ROUND(NVL(fs.free_mb,0),0) AS free_mb,
       ROUND((df.total_mb - NVL(fs.free_mb,0))/df.total_mb*100,1) AS
pct_used
FROM
  (SELECT tablespace_name, SUM(bytes)/1048576 total_mb
   FROM dba_data_files GROUP BY tablespace_name) df
LEFT JOIN
  (SELECT tablespace_name, SUM(bytes)/1048576 free_mb
   FROM dba_free_space GROUP BY tablespace_name) fs
ON df.tablespace_name = fs.tablespace_name
ORDER BY pct_used DESC NULLS LAST;
```

💡 **TIP:** Save the tablespace usage query above in a file called space_check.sql in /home/oracle/scripts/. Run it every morning as part of your daily health check.

✅ Lab Completion Checklist

- ☑ Created a tablespace with AUTOEXTEND enabled
- ☑ Added a second datafile to an existing tablespace

- ☑ Resized a datafile using ALTER DATABASE DATAFILE
- ☑ Can query DBA_DATA_FILES and DBA_FREE_SPACE to monitor space

🔬 LAB 08 Work with Tables, Indexes, and SQL Queries

⏱ Time: 45 min 📶 Level: Beginner

📌 **WHAT YOU WILL DO:** Create sample tables, insert data, create different types of indexes, run queries using the execution plan to see how indexes help performance.

🎯 **WHY THIS MATTERS:** Indexes are one of the most common interview topics. Seeing the execution plan change from 'TABLE ACCESS FULL' to 'INDEX RANGE SCAN' makes the concept real.

🏆 **WHAT YOU WILL LEARN:** CREATE INDEX, how to check execution plans with EXPLAIN PLAN and DBMS_XPLAN, and the visible difference indexes make on query speed.

```
-- Connect to PDB and create a test schema:
sqlplus system/SysPass123!@localhost:1521/orclpdb

CREATE USER testdba IDENTIFIED BY Test123!
DEFAULT TABLESPACE USERS QUOTA UNLIMITED ON USERS;
GRANT CONNECT, RESOURCE TO testdba;

-- Connect as testdba:
-- sqlplus testdba/Test123!@localhost:1521/orclpdb

-- Create a table with 100,000 rows (simulates real data volume):
CREATE TABLE orders (
  order_id    NUMBER PRIMARY KEY,
  customer_id NUMBER,
  order_date  DATE,
  amount      NUMBER(10,2),
  status      VARCHAR2(20)
);

-- Insert 100,000 rows (run this PL/SQL block):
BEGIN
  FOR i IN 1..100000 LOOP
```

```
    INSERT INTO orders VALUES (
      i,
      DBMS_RANDOM.VALUE(1,1000),
      SYSDATE - DBMS_RANDOM.VALUE(1,365),
      ROUND(DBMS_RANDOM.VALUE(10,10000),2),
      CASE MOD(i,3) WHEN 0 THEN 'PENDING'
                    WHEN 1 THEN 'SHIPPED' ELSE 'DELIVERED' END
    );
    IF MOD(i,1000)=0 THEN COMMIT; END IF;
  END LOOP;
  COMMIT;
END;
/

-- Gather statistics so optimizer knows about the data:
EXEC DBMS_STATS.GATHER_TABLE_STATS('TESTDBA','ORDERS');
```

Now test with and without index:

```
-- Check execution plan WITHOUT index (Full Table Scan):
EXPLAIN PLAN FOR
  SELECT * FROM orders WHERE customer_id = 500;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Look for: 'TABLE ACCESS FULL' -- Oracle reads ALL 100,000 rows!

-- Now create an index on customer_id:
CREATE INDEX idx_orders_cust ON orders(customer_id);

-- Check execution plan WITH index:
EXPLAIN PLAN FOR
  SELECT * FROM orders WHERE customer_id = 500;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Now look for: 'INDEX RANGE SCAN' -- Oracle uses the index!
-- Cost drops dramatically (e.g. from Cost=200 to Cost=4)

-- Test Bitmap index on low-cardinality column (status has only 3 values):
```

```
CREATE BITMAP INDEX idx_orders_status ON orders(status);

-- Rebuild an index:
ALTER INDEX idx_orders_cust REBUILD;

-- Make an index invisible (to test removing it safely):
ALTER INDEX idx_orders_cust INVISIBLE;

-- Make it visible again:
ALTER INDEX idx_orders_cust VISIBLE;
```

⭐ **IMPORTANT:** The EXPLAIN PLAN command is your best friend for performance tuning. Always run it before and after creating an index to confirm it actually gets used.

✅ Lab Completion Checklist

- ☑ Created a 100,000 row table and gathered statistics
- ☑ Saw TABLE ACCESS FULL in execution plan (no index)
- ☑ Created B-Tree index and saw INDEX RANGE SCAN in plan
- ☑ Created Bitmap index on low-cardinality column
- ☑ Practised REBUILD and INVISIBLE index operations

🔬 LAB 09 Manage Redo Logs and Enable Archive Log Mode

⏱ Time: 40 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Add redo log groups, multiplex redo logs to two locations, enable ARCHIVELOG mode, and verify archived logs are being created.

🎯 **WHY THIS MATTERS:** ARCHIVELOG mode is mandatory for hot backups and point-in-time recovery on production databases. Enabling it is a key DBA skill tested in every interview.

🏆 **WHAT YOU WILL LEARN:** Redo log architecture, how to add/multiplex redo groups, how to switch to ARCHIVELOG mode, and where archived logs go.

```
sqlplus / as sysdba

-- Check current redo log status:
SELECT group#, members, bytes/1024/1024 AS size_mb, status
FROM v$log;
```

```
-- See actual redo log files:
SELECT group#, member FROM v$logfile ORDER BY group#, member;

-- Add a new redo log group (Group 4) with 2 members (multiplexing):
ALTER DATABASE ADD LOGFILE GROUP 4 (
  '/u02/oradata/ORCL/redo04a.log',
  '/u02/oracle/FRA/ORCL/redo04b.log'
) SIZE 200M;

-- Add a member (multiplexed copy) to existing Group 1:
ALTER DATABASE ADD LOGFILE MEMBER
  '/u02/oracle/FRA/ORCL/redo01b.log' TO GROUP 1;

-- Verify multiplexing:
SELECT group#, member, status FROM v$logfile ORDER BY group#;

-- Check current archive mode:
SELECT log_mode FROM v$database;
-- Currently: NOARCHIVELOG
```

Enable ARCHIVELOG Mode (requires restart)

```
-- Set archive log destination:
ALTER SYSTEM SET log_archive_dest_1 =
'LOCATION=/u02/oracle/FRA/ORCL/archivelog'
SCOPE=SPFILE;

-- Step 1: Shutdown cleanly:
SHUTDOWN IMMEDIATE;

-- Step 2: Mount (not open):
STARTUP MOUNT;

-- Step 3: Enable ARCHIVELOG:
ALTER DATABASE ARCHIVELOG;

-- Step 4: Open the database:
ALTER DATABASE OPEN;
```

```
-- Step 5: Verify:
SELECT log_mode FROM v$database;
-- Should now show: ARCHIVELOG

-- Force a log switch to create first archive log:
ALTER SYSTEM SWITCH LOGFILE;
ALTER SYSTEM SWITCH LOGFILE;

-- Verify archive logs were created:
SELECT sequence#, name, applied FROM v$archived_log
ORDER BY sequence# DESC FETCH FIRST 5 ROWS ONLY;

-- Also check on filesystem:
-- (In a Linux terminal):
ls -lh /u02/oracle/FRA/ORCL/archivelog/
```

⚠️ **WARNING:** Once you enable ARCHIVELOG mode, archive logs accumulate on disk. Always monitor /u02/oracle/FRA and delete old logs after successful RMAN backup. A full archive destination stops the database!

✅ Lab Completion Checklist

- ☑ Viewed and added new redo log groups with the ALTER DATABASE command
- ☑ Multiplexed redo logs to two different disk locations
- ☑ Successfully switched database from NOARCHIVELOG to ARCHIVELOG mode
- ☑ Verified archived log files are being created in the destination folder

🔬 LAB 10 Control Files — Multiplex, Backup, and Restore

⏱ Time: 40 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Add a second copy of the control file to a different location (multiplexing), backup the control file, and practise restoring from a backup control file.

🎯 **WHY THIS MATTERS:** Control files are the most critical Oracle files — losing all copies means manual recovery. Multiplexing them to 3 locations is a production best practice every DBA must follow.

🏆 **WHAT YOU WILL LEARN:** How to multiplex control files, create a control file backup (trace file), and restore a lost control file.

```
sqlplus / as sysdba

-- See current control file locations:
SELECT name FROM v$controlfile;

-- See current SPFILE setting:
SHOW PARAMETER control_files;

-- Add a second control file location:
-- Step 1: Update SPFILE to add second location:
ALTER SYSTEM SET control_files =
  '/u02/oradata/ORCL/control01.ctl',
  '/u02/oracle/FRA/ORCL/control02.ctl'
SCOPE=SPFILE;

-- Step 2: Shutdown cleanly:
SHUTDOWN IMMEDIATE;

-- Step 3: Copy the existing control file to the new location (as oracle
user in Linux):
-- cp /u02/oradata/ORCL/control01.ctl /u02/oracle/FRA/ORCL/control02.ctl

-- Step 4: Start the database:
STARTUP;

-- Verify both control files are now active:
SELECT name FROM v$controlfile;

-- Create a text backup of control file (trace file):
-- This creates a SQL script you can use to recreate the control file if
all copies are lost:
ALTER DATABASE BACKUP CONTROLFILE TO TRACE
  AS '/backup/control_backup.sql' REUSE NORESETLOGS;

-- Also create a binary backup:
ALTER DATABASE BACKUP CONTROLFILE TO '/backup/control01_bkp.ctl';

-- View the trace file you created:
```

```
-- (In Linux terminal): cat /backup/control_backup.sql
```

💡 **TIP:** In production, always have control files in at least 3 different locations on different disks. If all control files are lost, you need RMAN backup or the trace file to rebuild — which can take hours.

✅ Lab Completion Checklist

☑ Viewed current control file locations from V$CONTROLFILE
☑ Added a second control file copy (multiplexed) by modifying SPFILE
☑ Created a text backup of control file using ALTER DATABASE BACKUP CONTROLFILE TO TRACE
☑ Created a binary backup of the control file

PHASE 3

Backup, Recovery & Data Movement

Labs 11 – 14 | Estimated Duration: 3 Hours

🔬 LAB 11 Configure RMAN and Run Your First Backup

⏱ Time: 50–60 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Configure RMAN retention policy, Fast Recovery Area, run a full database backup, and verify the backup was successful.

🎯 **WHY THIS MATTERS:** RMAN backup is asked in EVERY DBA interview. Running your first real RMAN backup and seeing it succeed builds enormous confidence.

🏆 **WHAT YOU WILL LEARN:** RMAN configuration parameters, CONFIGURE commands, BACKUP DATABASE PLUS ARCHIVELOG, verifying backups with LIST BACKUP.

Part A — Configure RMAN

```
# Connect to RMAN as oracle user on Linux:
rman target /

-- Check current RMAN configuration:
SHOW ALL;

-- Set retention policy (keep 7 days of backups):
CONFIGURE RETENTION POLICY TO RECOVERY WINDOW OF 7 DAYS;

-- Configure automatic backup of control file and SPFILE:
CONFIGURE CONTROLFILE AUTOBACKUP ON;
CONFIGURE CONTROLFILE AUTOBACKUP FORMAT FOR DEVICE TYPE DISK
  TO '/backup/rman/%F';

-- Set backup location (if not using FRA):
CONFIGURE CHANNEL DEVICE TYPE DISK
  FORMAT '/backup/rman/%U';

-- Enable backup compression:
CONFIGURE COMPRESSION ALGORITHM 'BASIC';
```

```
-- Show updated configuration:
SHOW ALL;
```

Part B — Run Full Database Backup

```
-- Run a full backup including archived logs (the most common production backup):
BACKUP AS COMPRESSED BACKUPSET DATABASE PLUS ARCHIVELOG
  TAG 'FULL_BACKUP_LAB11';

-- This will take 5-15 minutes. Watch the output for progress.
-- Look for: 'Finished backup at...' at the end.

-- Also run incremental level 0 (baseline for incrementals):
BACKUP INCREMENTAL LEVEL 0 DATABASE
  TAG 'INC0_BACKUP_LAB11';
```

Part C — Verify the Backup

```
-- List all backups RMAN knows about:
LIST BACKUP SUMMARY;

-- List backup details:
LIST BACKUP;

-- Cross-check: verify backup files still exist on disk:
CROSSCHECK BACKUP;

-- Verify backup is readable (validate without restoring):
RESTORE DATABASE VALIDATE;

-- Check RMAN backup history from SQL*Plus:
-- (In another terminal as oracle user)
sqlplus / as sysdba
SELECT start_time, end_time, status, input_type
FROM v$rman_backup_job_details
ORDER BY start_time DESC FETCH FIRST 5 ROWS ONLY;
```

✅ Lab Completion Checklist

☑ RMAN is configured with 7-day retention policy and automatic control file backup
☑ Full compressed database backup completed successfully including archive logs
☑ LIST BACKUP SUMMARY shows the backup and its details
☑ CROSSCHECK confirms all backup files exist on disk

🔬 LAB 12 RMAN Point-in-Time Recovery — Recover a Deleted Table

⏱ Time: 60 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Simulate a disaster: delete all data from a table and COMMIT it. Then use RMAN Point-in-Time Recovery (PITR) to restore the database to before the deletion.

🎯 **WHY THIS MATTERS:** PITR is the most powerful recovery technique. Demonstrating it confidently in an interview shows you are a real production DBA, not just a theory DBA.

🏆 **WHAT YOU WILL LEARN:** Full RMAN PITR workflow: RUN block with SET UNTIL TIME, RESTORE DATABASE, RECOVER DATABASE, OPEN RESETLOGS.

Part A — Create the Scenario (simulate the disaster)

```
sqlplus testdba/Test123!@localhost:1521/orclpdb

-- First, note the current time:
SELECT TO_CHAR(SYSDATE,'YYYY-MM-DD HH24:MI:SS') AS now FROM dual;
-- WRITE THIS TIME DOWN! e.g. 2026-04-10 15:30:00

-- Verify data exists:
SELECT COUNT(*) FROM orders;   -- Should show 100000

-- DISASTER: Delete everything and commit:
DELETE FROM orders;
COMMIT;

SELECT COUNT(*) FROM orders;   -- Now shows 0 -- data is GONE!

-- Note the current time again (time AFTER the disaster):
SELECT TO_CHAR(SYSDATE,'YYYY-MM-DD HH24:MI:SS') AS disaster_time FROM
dual;
```

Part B — Perform Point-in-Time Recovery

For CDB/PDB PITR, we recover the PDB specifically. This is a key Oracle 19c skill:

```
-- As SYSDBA, close the PDB first:
sqlplus / as sysdba
ALTER PLUGGABLE DATABASE orclpdb CLOSE IMMEDIATE;

-- In RMAN:
rman target /

-- Perform PDB Point-in-Time Recovery:
-- Replace '2026-04-10 15:30:00' with YOUR noted time (BEFORE the delete)
RUN {
  SET UNTIL TIME "TO_DATE('2026-04-10 15:30:00','YYYY-MM-DD HH24:MI:SS')";
  RESTORE PLUGGABLE DATABASE orclpdb;
  RECOVER PLUGGABLE DATABASE orclpdb;
  ALTER PLUGGABLE DATABASE orclpdb OPEN RESETLOGS;
}

-- After recovery, verify data is back:
sqlplus testdba/Test123!@localhost:1521/orclpdb
SELECT COUNT(*) FROM orders;
-- Should show 100000 again!
```

📝 **NOTE:** OPEN RESETLOGS creates a new database incarnation. After RESETLOGS, you must take a new RMAN full backup immediately because old backups cannot be used after this point.

```
-- IMPORTANT: Take a new full backup after RESETLOGS:
rman target /
BACKUP DATABASE PLUS ARCHIVELOG;
```

✅ Lab Completion Checklist

- ☑ Successfully simulated a disaster (bulk delete with COMMIT)
- ☑ Performed PDB-level Point-in-Time Recovery using RMAN
- ☑ Verified all 100,000 rows are restored after recovery
- ☑ Took a new full backup after RESETLOGS

🔬 LAB 13 Oracle Flashback — Fast Recovery Without RMAN

⏱ Time: 40 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Use Oracle Flashback features to recover accidentally deleted/modified rows WITHOUT needing RMAN restore. Flashback is faster and requires no downtime.

🎯 **WHY THIS MATTERS:** Flashback is the fastest recovery tool for small mistakes. In interviews you must know BOTH Flashback AND RMAN and when to use which.

🏆 **WHAT YOU WILL LEARN:** Flashback Query (AS OF TIMESTAMP), Flashback Table, Flashback Drop (Recycle Bin), and Flashback Database.

```
sqlplus testdba/Test123!@localhost:1521/orclpdb

-- ── FLASHBACK QUERY ──
-- See what data looked like 15 minutes ago:
SELECT COUNT(*) FROM orders
AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '15' MINUTE);

-- See specific rows that were deleted 10 minutes ago:
SELECT * FROM orders
AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '10' MINUTE)
WHERE customer_id = 500;

-- ── FLASHBACK TABLE ──
-- Enable row movement on the table (required for flashback table):
ALTER TABLE orders ENABLE ROW MOVEMENT;

-- Delete some rows to create a problem:
DELETE FROM orders WHERE customer_id = 500;
COMMIT;
SELECT COUNT(*) FROM orders WHERE customer_id=500; -- Shows 0

-- Flashback the entire table to 5 minutes ago:
FLASHBACK TABLE orders
TO TIMESTAMP (SYSTIMESTAMP - INTERVAL '5' MINUTE);

SELECT COUNT(*) FROM orders WHERE customer_id=500; -- Rows are back!
```

```
-- ── FLASHBACK DROP (RECYCLE BIN) ──
-- Drop a table:
DROP TABLE orders;

-- Check Recycle Bin:
SELECT object_name, original_name, type, droptime FROM recyclebin;

-- Restore dropped table:
FLASHBACK TABLE orders TO BEFORE DROP;

SELECT COUNT(*) FROM orders;  -- Table is back!

-- Purge (permanently delete) from Recycle Bin:
-- PURGE TABLE orders;   -- Purge specific table
-- PURGE RECYCLEBIN;     -- Purge all your objects
```

💡 **TIP:** Flashback works based on UNDO data. If UNDO_RETENTION is set to 900 seconds (15 minutes), you can flashback up to 15 minutes back. For longer windows, increase UNDO_RETENTION: ALTER SYSTEM SET UNDO_RETENTION=3600 SCOPE=BOTH;

✅ Lab Completion Checklist

- ☑ Used Flashback Query (AS OF TIMESTAMP) to see historical data
- ☑ Used Flashback Table to restore deleted rows without downtime
- ☑ Used Flashback Drop to recover a dropped table from Recycle Bin
- ☑ Understand the difference between Flashback and RMAN PITR

🔬 LAB 14 Data Pump — Export (expdp) and Import (impdp)

⏱ Time: 45 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Use Oracle Data Pump to export a schema to a dump file, then import it into a different schema — the standard way to move data between databases.

🎯 **WHY THIS MATTERS:** Data Pump is used daily for database refreshes, migrations, creating test copies, and moving data. Every mid-level DBA must know expdp/impdp commands.

🏆 **WHAT YOU WILL LEARN:** Creating a DATA PUMP DIRECTORY, expdp full schema export, impdp schema import with REMAP_SCHEMA, and monitoring long-running Data Pump jobs.

```
sqlplus / as sysdba
ALTER SESSION SET CONTAINER = ORCLPDB;

-- Step 1: Create a directory for dump files (must exist on OS too):
CREATE OR REPLACE DIRECTORY dpump_dir AS '/backup/datapump';
GRANT READ, WRITE ON DIRECTORY dpump_dir TO testdba;

-- Also create it on OS (as oracle user in terminal):
-- mkdir -p /backup/datapump

-- Step 2: Export the TESTDBA schema:
-- (Run in Linux terminal as oracle user, NOT in SQL*Plus)
expdp testdba/Test123!@localhost:1521/orclpdb \
  SCHEMAS=TESTDBA \
  DIRECTORY=DPUMP_DIR \
  DUMPFILE=testdba_export.dmp \
  LOGFILE=testdba_export.log \
  COMPRESSION=ALL

-- This creates: /backup/datapump/testdba_export.dmp

-- Step 3: Create a new target schema to import into:
sqlplus / as sysdba
ALTER SESSION SET CONTAINER = ORCLPDB;
CREATE USER testdba2 IDENTIFIED BY Test123!
DEFAULT TABLESPACE USERS QUOTA UNLIMITED ON USERS;
GRANT CONNECT, RESOURCE TO testdba2;
GRANT READ,WRITE ON DIRECTORY dpump_dir TO testdba2;

-- Step 4: Import into testdba2 schema (REMAP remaps source schema to
target):
impdp system/SysPass123!@localhost:1521/orclpdb \
  DIRECTORY=DPUMP_DIR \
  DUMPFILE=testdba_export.dmp \
```

```
  LOGFILE=testdba_import.log \
  REMAP_SCHEMA=TESTDBA:TESTDBA2

-- Verify data in new schema:
sqlplus testdba2/Test123!@localhost:1521/orclpdb
SELECT COUNT(*) FROM orders;  -- Should show same 100,000 rows
```

Export / Import — Other Useful Options

```
-- Export only specific tables:
expdp testdba/Test123!@localhost:1521/orclpdb \
  TABLES=TESTDBA.ORDERS \
  DIRECTORY=DPUMP_DIR \
  DUMPFILE=orders_only.dmp \
  LOGFILE=orders_exp.log

-- Export entire database (as SYSDBA):
expdp \'/ as sysdba\' \
  FULL=Y \
  DIRECTORY=DPUMP_DIR \
  DUMPFILE=fulldb_%U.dmp \
  LOGFILE=fulldb_exp.log \
  PARALLEL=4

-- Monitor a running Data Pump job:
-- In SQL*Plus: SELECT job_name, state, degree FROM dba_datapump_jobs;
-- Or attach to a running job: expdp ATTACH=SYS_EXPORT_SCHEMA_01
```

✅ Lab Completion Checklist

☑ Created DATA PUMP DIRECTORY object in the database and matching OS folder
☑ Exported TESTDBA schema to a .dmp file using expdp
☑ Imported into a different schema (TESTDBA2) using REMAP_SCHEMA
☑ Know how to export specific tables and how to monitor running jobs

PHASE 4

Performance Tuning & Monitoring

Labs 15 – 17 | Estimated Duration: 2–3 Hours

🔬 LAB 15 Generate AWR Report & Analyse ADDM Findings

⏱ Time: 45–60 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Generate an AWR (Automatic Workload Repository) report for your database, read its key sections, and review the automatic ADDM diagnosis.

🎯 **WHY THIS MATTERS:** AWR and ADDM are the first tools you open when performance problems are reported. Being able to read an AWR report fluently is what separates mid-level DBAs.

🏆 **WHAT YOU WILL LEARN:** How AWR snapshots work, how to generate the report, how to read Top 5 Wait Events and Top SQL sections, and how to use ADDM findings.

```
sqlplus / as sysdba


-- First: generate some database activity to make the AWR report
meaningful.
-- Run this workload in the ORCLPDB:
ALTER SESSION SET CONTAINER = ORCLPDB;


BEGIN
  FOR i IN 1..50000 LOOP
    EXECUTE IMMEDIATE
      'SELECT COUNT(*) FROM testdba.orders WHERE customer_id = ' || i;
  END LOOP;
END;
/
-- (This creates many hard parses -- a common performance problem)


-- Take a manual AWR snapshot to capture this activity:
EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT();


-- Run more activity...
BEGIN
```

```
  FOR i IN 1..50000 LOOP
    EXECUTE IMMEDIATE
      'SELECT /*+ INDEX(o idx_orders_cust) */ * FROM testdba.orders o
       WHERE customer_id = ' || i || ' AND rownum=1';
  END LOOP;
END;
/

-- Take another snapshot:
EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT();

-- List available AWR snapshots:
SELECT snap_id, begin_interval_time, end_interval_time
FROM dba_hist_snapshot
ORDER BY snap_id DESC FETCH FIRST 10 ROWS ONLY;
-- Note the last 2 snap_id numbers (e.g. 12 and 13)
```

Generate the AWR Report

```
-- Run the AWR report script:
@$ORACLE_HOME/rdbms/admin/awrrpt.sql

-- The script will ask:
-- 1. Report format: html  (type: html and press Enter)
-- 2. Number of days: 1
-- 3. Begin snapshot ID: [type your first snap_id]
-- 4. End snapshot ID: [type your second snap_id]
-- 5. Report name: awrrpt_lab15.html (or press Enter for default)

-- The HTML file is created in your current directory.
-- Copy it to your Windows machine with WinSCP and open in Chrome.

-- What to look for in the report:
-- 1. 'Top 5 Timed Foreground Events' -- where Oracle spent most time
-- 2. 'SQL ordered by Elapsed Time' -- slowest queries
-- 3. 'SQL ordered by Executions' -- most frequently run queries
-- 4. 'Instance Efficiency Percentages' -- Buffer Hit % should be >95%
-- 5. 'Redo Log Switches' -- high count means redo logs are too small
```

Check ADDM (Automatic Diagnosis)

```
-- ADDM automatically analyses AWR and lists findings:
@$ORACLE_HOME/rdbms/admin/addmrpt.sql

-- ADDM shows findings like:
-- FINDING 1: Hard parse activity was high (4,521 hard parses/sec)
--   Recommendation: Use bind variables in application queries
-- FINDING 2: SQL statement with sql_id=x used 45% of DB time
--   Recommendation: Tune this query / add index

-- You can also query ADDM findings from SQL:
SELECT finding_name, type, message FROM dba_advisor_findings
WHERE task_name LIKE 'ADDM%'
ORDER BY impact_db_time DESC FETCH FIRST 5 ROWS ONLY;
```

✅ Lab Completion Checklist

☑ Created manual AWR snapshots and generated an AWR HTML report
☑ Can identify Top 5 Wait Events in an AWR report
☑ Can find the Top SQL by elapsed time and execution count
☑ Generated an ADDM report and read its performance recommendations

🔬 LAB 16 SQL Performance — Find Slow Queries & Fix Them

⏱ Time: 45 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Find the top resource-consuming SQL queries using V$SQL, check their execution plans, identify performance issues, and fix them using indexes and hints.

🎯 **WHY THIS MATTERS:** SQL tuning is 70% of what performance tuning work involves. Being able to find and fix a slow query is the most valuable skill a mid-level DBA has.

🏆 **WHAT YOU WILL LEARN:** V$SQL view, reading execution plans from V$SQL_PLAN, SQL hints (INDEX hint, FULL hint), and gathering fresh optimizer statistics.

```
sqlplus / as sysdba
```

```
-- Find Top 10 SQL by elapsed time:
SELECT sql_id, ROUND(elapsed_time/1000000,2) elapsed_sec,
       executions,
ROUND(elapsed_time/DECODE(executions,0,1,executions)/1000000,2) avg_sec,
       ROUND(cpu_time/1000000,2) cpu_sec,
       SUBSTR(sql_text,1,70) AS sql_preview
FROM v$sql
WHERE executions > 0
ORDER BY elapsed_time DESC
FETCH FIRST 10 ROWS ONLY;

-- Find Top SQL by buffer gets (logical reads -- means lots of data
scanning):
SELECT sql_id, buffer_gets, executions,
       ROUND(buffer_gets/DECODE(executions,0,1,executions),0)
bufgets_per_exec,
       SUBSTR(sql_text,1,70) AS sql_preview
FROM v$sql
WHERE executions > 0
ORDER BY buffer_gets DESC
FETCH FIRST 10 ROWS ONLY;

-- Find the execution plan for a specific sql_id:
-- (Replace 'abc123xyz' with an actual sql_id from above query)
SELECT * FROM TABLE(
  DBMS_XPLAN.DISPLAY_CURSOR('abc123xyz', NULL, 'ALLSTATS LAST'));
```

Fix a Slow Query Using the SQL Tuning Advisor

```
-- Oracle's built-in SQL Tuning Advisor can automatically suggest fixes:
DECLARE
  l_task_name VARCHAR2(30);
BEGIN
  l_task_name := DBMS_SQLTUNE.CREATE_TUNING_TASK(
    sql_text  => 'SELECT * FROM testdba.orders WHERE customer_id = :cust',
    task_name => 'TUNE_ORDERS_QUERY');
  DBMS_SQLTUNE.EXECUTE_TUNING_TASK(task_name => 'TUNE_ORDERS_QUERY');
END;
/
```

```
-- See the recommendations:
SELECT DBMS_SQLTUNE.REPORT_TUNING_TASK('TUNE_ORDERS_QUERY')
FROM DUAL;
-- The advisor may say: 'Create index on customer_id -- benefit: 97%'

-- Always update statistics when performance changes after large data
loads:
EXEC DBMS_STATS.GATHER_SCHEMA_STATS('TESTDBA', DEGREE=>4);

-- Or gather stats for a specific table:
EXEC DBMS_STATS.GATHER_TABLE_STATS('TESTDBA','ORDERS',CASCADE=>TRUE);
-- CASCADE=>TRUE also gathers stats on all indexes on that table
```

✅ Lab Completion Checklist

☑ Found top SQL queries by elapsed time and buffer gets using V$SQL
☑ Read an execution plan from V$SQL_PLAN using DBMS_XPLAN.DISPLAY_CURSOR
☑ Used SQL Tuning Advisor (DBMS_SQLTUNE) to get automatic recommendations
☑ Gathered fresh optimizer statistics using DBMS_STATS

🔬 LAB 17 Sessions, Locks & Blocking — Find & Fix

⏱ Time: 40 min 📶 Level: **Intermediate**

📌 **WHAT YOU WILL DO:** Simulate a locking scenario where one session blocks another, identify the blocking session using V$ views, and kill it to resolve the issue.

🎯 **WHY THIS MATTERS:** Blocking sessions cause application timeouts and user complaints. This is a scenario every DBA faces weekly in production.

🏆 **WHAT YOU WILL LEARN:** V$SESSION, V$LOCK views, identifying blocking chains, ALTER SYSTEM KILL SESSION, and when to kill vs when to wait.

Part A — Create a Lock Scenario

Open TWO separate SQL*Plus sessions simultaneously (two terminal windows):

```
-- === SESSION 1 (Terminal 1) ===
sqlplus testdba/Test123!@localhost:1521/orclpdb
```

```
-- Update a row WITHOUT committing (creates a row lock):
UPDATE orders SET amount = 9999 WHERE order_id = 1;
-- DO NOT type COMMIT yet! Leave this session open.



-- === SESSION 2 (Terminal 2) ===
sqlplus testdba/Test123!@localhost:1521/orclpdb

-- Try to update the SAME row -- this will HANG (blocked):
UPDATE orders SET amount = 8888 WHERE order_id = 1;
-- This session will just hang... waiting for Session 1 to commit or
rollback.
```

Part B — Find the Blocking Session (open 3rd terminal as sysdba)

```
sqlplus / as sysdba

-- Find blocking sessions:
SELECT s.sid, s.serial#, s.username, s.status,
       s.blocking_session AS blocked_by,
       s.seconds_in_wait, s.wait_class, s.event,
       s.machine, s.program
FROM v$session s
WHERE s.blocking_session IS NOT NULL
ORDER BY s.seconds_in_wait DESC;

-- Find what SQL the blocked session is waiting on:
SELECT s.sid, s.serial#, s.blocking_session,
       q.sql_text AS waiting_sql
FROM v$session s
JOIN v$sql q ON s.sql_id = q.sql_id
WHERE s.blocking_session IS NOT NULL;

-- Show the full lock chain:
SELECT l.sid, l.type, l.mode_held, l.mode_requested,
       s.username, s.status, s.machine
FROM v$lock l JOIN v$session s ON l.sid = s.sid
WHERE l.block = 1 OR l.request > 0
ORDER BY l.sid;
```

Part C — Kill the Blocking Session

```
-- Kill the blocking session (replace SID and SERIAL# with actual values):
ALTER SYSTEM KILL SESSION 'sid,serial#' IMMEDIATE;

-- Example: ALTER SYSTEM KILL SESSION '27,15432' IMMEDIATE;

-- After killing, Session 2 should either complete or get an error.
-- Session 1's transaction will be ROLLED BACK automatically.

-- Verify no more blocked sessions:
SELECT sid, blocking_session FROM v$session
WHERE blocking_session IS NOT NULL;
-- Should return no rows now.
```

⚠️ **WARNING:** In production, DO NOT kill sessions without first checking who owns it and informing the application team. Killing a session rolls back its transaction — this may cause data inconsistency in the application.

✅ Lab Completion Checklist

- ☑ Created a real lock scenario with two sessions competing for the same row
- ☑ Identified the blocking session and blocked session using V$SESSION and V$LOCK
- ☑ Found what SQL the blocked session was waiting to run
- ☑ Successfully killed the blocking session using ALTER SYSTEM KILL SESSION

PHASE 5

Advanced DBA Topics — Patching, Data Guard & Monitoring

Labs 18 – 20 | Estimated Duration: 3–4 Hours

🔬 LAB 18 Apply an Oracle Patch Using OPatch

⏱ Time: 45–60 min 📶 Level: Advanced

📌 **WHAT YOU WILL DO:** Download a sample Oracle patch, check for conflicts, apply it using OPatch, run datapatch, and verify the patch was applied successfully.

🎯 **WHY THIS MATTERS:** Patching is a critical skill asked in every mid-level DBA interview. Knowing the exact steps and commands in the correct order is what sets you apart.

🏆 **WHAT YOU WILL LEARN:** OPatch lsinventory, prereq checks, opatch apply workflow, datapatch, and post-patch validation.

⭐ **IMPORTANT:** For this lab, use any available patch from Oracle Support (requires free Oracle account). You can download a one-off patch to practice the steps. The patch NUMBER will differ but the PROCESS is always the same.

```
# ── PRE-PATCH STEPS ──

# 1. Check current OPatch version:
$ORACLE_HOME/OPatch/opatch version
# Should be 12.2.0.1.x or higher. If not, update OPatch first.

# 2. List all patches currently applied:
$ORACLE_HOME/OPatch/opatch lsinventory
# Note the patch IDs that are already installed.

# 3. Check OS disk space (patch needs at least 10 GB free in Oracle Home
area):
df -h /u01

# 4. BACKUP Oracle Home before patching:
tar -czf /backup/oracle_home_pre_patch_$(date +%Y%m%d).tar.gz $ORACLE_HOME
# This takes 15-30 minutes and creates a ~2-3 GB file
```

```
# 5. Download the patch from support.oracle.com
#    Unzip it: unzip p<patch_number>_190000_Linux-x86-64.zip -d
/backup/patch/
cd /backup/patch/<patch_directory>

# 6. Check for patch conflicts BEFORE applying:
$ORACLE_HOME/OPatch/opatch prereq CheckConflictAgainstOHWithDetail \
  -ph /backup/patch/<patch_directory>
# Look for: 'Prereq "checkConflictAgainstOHWithDetail" passed.'

# ── APPLY THE PATCH ──

# 7. Stop the database and listener:
sqlplus / as sysdba
# SHUTDOWN IMMEDIATE;
# EXIT
lsnrctl stop

# 8. Apply the patch:
$ORACLE_HOME/OPatch/opatch apply
# When prompted: press Y to continue
# Wait for: 'OPatch succeeded.'

# ── POST-PATCH STEPS ──

# 9. Start the listener and database:
lsnrctl start
sqlplus / as sysdba
# STARTUP;

# 10. Run datapatch to apply database-level changes:
cd $ORACLE_HOME/OPatch
./datapatch -verbose
# Wait for: 'Patch installation complete.'

# 11. Compile any invalid objects:
sqlplus / as sysdba
```

```
# @?/rdbms/admin/utlrp.sql

# 12. Verify the patch is installed:
$ORACLE_HOME/OPatch/opatch lsinventory | grep <patch_number>

# 13. Query to verify from database:
SELECT patch_id, action, status, description
FROM dba_registry_sqlpatch
ORDER BY action_time DESC FETCH FIRST 5 ROWS ONLY;
```

📝 **NOTE:** If OPatch fails midway: do NOT restart the database. Check the OPatch log file in /u01/app/oraInventory/logs/ for the error. Fix the issue and re-run opatch apply — it will resume from where it failed.

✅ Lab Completion Checklist

- ☑ Know how to check current patch level with opatch lsinventory
- ☑ Can run prereq conflict check before applying any patch
- ☑ Know the complete patch workflow: backup, stop DB, apply, start DB, datapatch, validate
- ☑ Can verify applied patches using opatch lsinventory and DBA_REGISTRY_SQLPATCH

🔬 LAB 19 Set Up Oracle Data Guard — Physical Standby Database

⏱ Time: 90 min 📶 Level: Advanced

📌 **WHAT YOU WILL DO:** Create a second virtual machine (or use a second directory) to set up a Physical Standby database that receives and applies redo logs from the primary.

🎯 **WHY THIS MATTERS:** Data Guard is the most important HA feature in Oracle and is asked in almost all senior/mid-level DBA interviews. Setting it up once gives you deep understanding.

🏆 **WHAT YOU WILL LEARN:** Data Guard concepts in practice: primary setup, duplicate to standby with RMAN, log transport, and verifying redo apply.

⭐ **IMPORTANT:** For this lab, you will simulate a Data Guard setup on ONE server using two different Oracle homes and databases: ORCL (primary) and ORCLSTBY (standby). This is standard practice for training environments.

Part A — Prepare Primary Database (ORCL)

```
sqlplus / as sysdba

-- Step 1: Enable FORCE LOGGING (required for Data Guard):
ALTER DATABASE FORCE LOGGING;
SELECT force_logging FROM v$database;  -- Should show YES

-- Step 2: Enable Standby Redo Logs on primary:
-- (One more group than redo log groups -- if 3 redo groups, add 4 standby
groups)
ALTER DATABASE ADD STANDBY LOGFILE GROUP 5
  '/u02/oradata/ORCL/stby_redo05.log' SIZE 200M;
ALTER DATABASE ADD STANDBY LOGFILE GROUP 6
  '/u02/oradata/ORCL/stby_redo06.log' SIZE 200M;
ALTER DATABASE ADD STANDBY LOGFILE GROUP 7
  '/u02/oradata/ORCL/stby_redo07.log' SIZE 200M;
ALTER DATABASE ADD STANDBY LOGFILE GROUP 8
  '/u02/oradata/ORCL/stby_redo08.log' SIZE 200M;

-- Step 3: Set DB_UNIQUE_NAME and LOG_ARCHIVE_CONFIG:
ALTER SYSTEM SET db_unique_name='ORCL' SCOPE=SPFILE;
ALTER SYSTEM SET log_archive_config='DG_CONFIG=(ORCL,ORCLSTBY)'
SCOPE=BOTH;

-- Step 4: Set LOG_ARCHIVE_DEST for standby:
ALTER SYSTEM SET log_archive_dest_1=
  'LOCATION=/u02/oracle/FRA/ORCL/archivelog
VALID_FOR=(ALL_LOGFILES,ALL_ROLES)'
  SCOPE=BOTH;
ALTER SYSTEM SET log_archive_dest_2=
  'SERVICE=ORCLSTBY ASYNC VALID_FOR=(ONLINE_LOGFILES,PRIMARY_ROLE)
   DB_UNIQUE_NAME=ORCLSTBY'
  SCOPE=BOTH;

-- Step 5: Enable archivelog dest states:
ALTER SYSTEM SET log_archive_dest_state_1=ENABLE SCOPE=BOTH;
ALTER SYSTEM SET log_archive_dest_state_2=ENABLE SCOPE=BOTH;

-- Step 6: Set STANDBY_FILE_MANAGEMENT:
ALTER SYSTEM SET standby_file_management=AUTO SCOPE=BOTH;
```

```
-- Step 7: Set FAL parameters:
ALTER SYSTEM SET fal_server=ORCLSTBY SCOPE=BOTH;
ALTER SYSTEM SET fal_client=ORCL SCOPE=BOTH;
```

Part B — Create the Standby Database with RMAN DUPLICATE

```
-- Create directories for the standby database:
mkdir -p /u02/oradata/ORCLSTBY
mkdir -p /u02/oracle/FRA/ORCLSTBY

-- Create a pfile for standby (create /tmp/orclstby_init.ora):
cat > /tmp/orclstby_init.ora << 'EOF'
db_name=ORCL
db_unique_name=ORCLSTBY
db_file_name_convert='/u02/oradata/ORCL','/u02/oradata/ORCLSTBY'
log_file_name_convert='/u02/oradata/ORCL','/u02/oradata/ORCLSTBY'
control_files='/u02/oradata/ORCLSTBY/control01.ctl'
db_recovery_file_dest='/u02/oracle/FRA/ORCLSTBY'
db_recovery_file_dest_size=20G
audit_file_dest='/u01/app/oracle/admin/ORCLSTBY/adump'
EOF

-- Start standby in NOMOUNT using its pfile:
# Set ORACLE_SID=ORCLSTBY in the environment
export ORACLE_SID=ORCLSTBY
sqlplus / as sysdba
STARTUP NOMOUNT PFILE='/tmp/orclstby_init.ora';
EXIT

-- Duplicate from primary to standby using RMAN:
rman target sys/SysPass123!@ORCL auxiliary sys/SysPass123!@ORCLSTBY

DUPLICATE TARGET DATABASE FOR STANDBY
  FROM ACTIVE DATABASE
  DORECOVER
  SPFILE
    PARAMETER_VALUE_CONVERT 'ORCL','ORCLSTBY'
```

```
  NOFILENAMECHECK;
# This copies all data files from primary to standby -- takes 10-30
minutes
```

Part C — Verify Data Guard is Working

```
-- On PRIMARY -- check log transport status:
export ORACLE_SID=ORCL
sqlplus / as sysdba
SELECT dest_id, dest_name, status, target, archiver,
       schedule, destination FROM v$archive_dest
WHERE dest_id <= 2;

-- Force a log switch on primary:
ALTER SYSTEM SWITCH LOGFILE;

-- On STANDBY -- verify redo is being applied:
export ORACLE_SID=ORCLSTBY
sqlplus / as sysdba
SELECT sequence#, applied, name FROM v$archived_log
ORDER BY sequence# DESC FETCH FIRST 5 ROWS ONLY;

-- Check standby database role:
SELECT name, db_unique_name, database_role, open_mode FROM v$database;
-- Should show: DATABASE_ROLE = PHYSICAL STANDBY

-- Check MRP (Managed Recovery Process) is running:
SELECT process, status, sequence# FROM v$managed_standby;
-- Look for: MRP0 with status APPLYING_LOG
```

💡 **TIP:** For a real 2-node Data Guard setup (two VMs), you would add the second VM IP to /etc/hosts and create TNS entries in tnsnames.ora so the primary and standby can communicate over the network.

✅ Lab Completion Checklist

☑ Primary database configured with FORCE LOGGING, Standby Redo Logs, and LOG_ARCHIVE_DEST_2
☑ Standby database created using RMAN DUPLICATE FROM ACTIVE DATABASE
☑ Redo logs are being shipped from primary and applied on standby

☑ Standby shows DATABASE_ROLE = PHYSICAL STANDBY and MRP0 is APPLYING_LOG

🔬 LAB 20 Daily Monitoring — Alert Log, Health Check Scripts & OEM

⏱ Time: 45 min 📶 Level: Advanced

📌 **WHAT YOU WILL DO:** Build a daily DBA health check SQL script, learn to monitor the alert log, set up a simple automated monitoring check, and explore Oracle Enterprise Manager Express.

🎯 **WHY THIS MATTERS:** Proactive monitoring is what separates a reactive DBA (fixes problems after users scream) from a proactive DBA (catches problems before users notice). Proactive DBAs get promoted.

🏆 **WHAT YOU WILL LEARN:** Alert log monitoring, creating a reusable health check script, exploring OEM Express, and understanding how to automate monitoring.

Part A — Monitor the Alert Log

```
-- Query alert log from SQL*Plus (modern way):
sqlplus / as sysdba

-- Find ORA- errors from last 24 hours:
SELECT originating_timestamp, message_text
FROM v$diag_alert_ext
WHERE originating_timestamp > SYSDATE - 1
  AND message_text LIKE '%ORA-%'
ORDER BY originating_timestamp DESC
FETCH FIRST 20 ROWS ONLY;

-- Find the physical alert log file location:
SELECT value FROM v$diag_info WHERE name = 'Diag Trace';

-- View the alert log from Linux terminal (last 100 lines):
tail -100 $ORACLE_BASE/diag/rdbms/orcl/ORCL/trace/alert_ORCL.log

-- Watch alert log live (like a log stream):
tail -f $ORACLE_BASE/diag/rdbms/orcl/ORCL/trace/alert_ORCL.log
# Press Ctrl+C to stop watching
```

Part B — Create Your Daily Health Check Script

Save this as /home/oracle/scripts/daily_healthcheck.sql

```
-- daily_healthcheck.sql
-- Run this every morning: sqlplus / as sysdba
@/home/oracle/scripts/daily_healthcheck.sql

SET PAGESIZE 200 LINESIZE 200 FEEDBACK OFF
PROMPT ==============================================================
PROMPT  ORACLE DAILY HEALTH CHECK - Run at: &&_DATE
PROMPT ==============================================================

PROMPT --- 1. DATABASE STATUS ---
SELECT name, open_mode, log_mode, db_unique_name FROM v$database;

PROMPT --- 2. INSTANCE STATUS ---
SELECT instance_name, status, startup_time FROM v$instance;

PROMPT --- 3. TABLESPACE USAGE (Alert if > 80%) ---
SELECT df.tablespace_name,
       ROUND((df.total-NVL(fs.free,0))/df.total*100,1) AS pct_used
FROM (SELECT tablespace_name, SUM(bytes) total FROM dba_data_files GROUP
BY tablespace_name) df
LEFT JOIN (SELECT tablespace_name, SUM(bytes) free FROM dba_free_space
GROUP BY tablespace_name) fs
ON df.tablespace_name=fs.tablespace_name
WHERE ROUND((df.total-NVL(fs.free,0))/df.total*100,1) > 70
ORDER BY pct_used DESC;

PROMPT --- 4. LAST RMAN BACKUP STATUS ---
SELECT TO_CHAR(start_time,'DD-MON-YYYY HH24:MI') started,
       TO_CHAR(end_time,'DD-MON-YYYY HH24:MI') ended,
       status, input_type
FROM v$rman_backup_job_details
ORDER BY start_time DESC FETCH FIRST 3 ROWS ONLY;

PROMPT --- 5. INVALID OBJECTS ---
SELECT owner, object_type, COUNT(*) AS invalid_count
FROM dba_objects WHERE status='INVALID'
```

```
GROUP BY owner, object_type
ORDER BY 1,2;

PROMPT --- 6. BLOCKING SESSIONS ---
SELECT sid, serial#, username, blocking_session, seconds_in_wait
FROM v$session WHERE blocking_session IS NOT NULL
ORDER BY seconds_in_wait DESC;

PROMPT --- 7. FAILED SCHEDULER JOBS (Last 24 hrs) ---
SELECT job_name, log_date, status, error#
FROM dba_scheduler_job_run_details
WHERE status != 'SUCCEEDED'
  AND log_date > SYSDATE-1
ORDER BY log_date DESC;

PROMPT === HEALTH CHECK COMPLETE ===
EXIT;
```

Part C — Access Oracle Enterprise Manager (OEM) Express

```
-- Find OEM Express URL:
sqlplus / as sysdba
SELECT dbms_xdb_config.gethttpsport() AS https_port FROM dual;

-- If port is 0, enable OEM Express:
EXEC DBMS_XDB_CONFIG.SETGLOBALPORTENABLED(TRUE);
EXEC DBMS_XDB_CONFIG.SETHTTPSPORT(5500);

-- Access OEM Express from your browser:
-- URL: https://<your_vm_ip>:5500/em
-- Login: SYS / SysPass123! / as SYSDBA

-- Explore these sections in OEM Express:
-- 1. Performance Hub -- AWR and real-time performance charts
-- 2. Storage -- Tablespace usage with visual bars
-- 3. Security -- Users and their privileges
-- 4. Configuration -- All V$PARAMETER settings in a GUI
```

Part D — Set Up a Simple Cron Job for Monitoring

```
# As oracle user on Linux, set up a cron job to run health check daily at
8 AM:

crontab -e


# Add this line (saves output to log file):

0 8 * * * sqlplus / as sysdba @/home/oracle/scripts/daily_healthcheck.sql
\

  > /home/oracle/logs/healthcheck_$(date +\%Y\%m\%d).log 2>&1


# Verify the cron job was saved:

crontab -l


# Create the logs directory:

mkdir -p /home/oracle/logs
```

💡 **TIP:** Add this to your resume: 'Created automated daily health check scripts and configured cron jobs for proactive database monitoring.' This shows initiative and production mindset.

✅ Lab Completion Checklist

☑ Can query Oracle alert log for ORA- errors using V$DIAG_ALERT_EXT and tail command
☑ Created a reusable daily health check SQL script covering all key areas
☑ Can access and navigate OEM Express for visual database monitoring
☑ Set up a cron job to run health check automatically every morning

Congratulations — You Are Now a Practical Oracle DBA!

If you have completed all 20 labs, you now have more hands-on Oracle experience than many people who have been reading books for months. Real commands. Real database. Real skills.

🏆 What You Have Achieved

- ✅ Set up a full Oracle 19c lab environment on Linux (VirtualBox + Oracle Linux)
- ✅ Installed Oracle 19c Database software from scratch
- ✅ Created a CDB + PDB database using DBCA
- ✅ Mastered database startup/shutdown and listener management
- ✅ Explored Oracle architecture by querying real V$ views
- ✅ Created users, roles, profiles, and practised privilege management
- ✅ Managed tablespaces and datafiles — added space, monitored usage
- ✅ Created a 100,000-row table and proved how indexes improve performance
- ✅ Enabled ARCHIVELOG mode — the key to production backup/recovery
- ✅ Multiplexed control files and redo logs for protection
- ✅ Configured RMAN and ran full and incremental backups
- ✅ Performed Point-in-Time Recovery — recovered 100,000 deleted rows
- ✅ Used Oracle Flashback — recovered data without any downtime
- ✅ Used Data Pump (expdp/impdp) to export and import schemas
- ✅ Generated and read an AWR report and ADDM findings
- ✅ Found and fixed slow SQL queries using V$SQL and Tuning Advisor
- ✅ Simulated and resolved blocking session scenarios
- ✅ Walked through the complete Oracle patching workflow
- ✅ Configured a Physical Standby database using Data Guard
- ✅ Built a daily automated health check monitoring system

📚 What to Study Next

- Oracle Multitenant (CDB/PDB) deep-dive — Plug/unplug PDBs, PDB cloning
- Oracle ASM (Automatic Storage Management) — Disk groups, failure groups, rebalancing
- Oracle GoldenGate — Logical replication for migrations and zero-downtime upgrades
- Oracle RAC — Multi-node cluster setup (requires 2 VMs with 8+ GB RAM each)
- Oracle Cloud (OCI) — Autonomous Database, Base Database Service

- SQL Server DBA — AlwaysOn AG, SQL Server Agent, SSMS tooling
- Oracle Performance Tuning — Deeper AWR analysis, SQL Plan Baselines, Adaptive Plans

🔗 Trusted Reference Links

oracle-base.com — Best free Oracle tutorial site. All topics with copy-paste commands.
docs.oracle.com/en/database/oracle/oracle-database/19/ — Official Oracle 19c documentation.
support.oracle.com — Oracle My Support (free account) for patches and bug reports.
asktom.oracle.com — Tom Kyte's Q&A site. The best Oracle DBA wisdom resource online.
dbagenesis.com — DBA Genesis tutorials. Practical guides for all Oracle admin tasks.

Keep practising. Keep breaking things. Keep fixing them. That is how DBAs grow. 💪